

Raport z testów

Aplikacja: Quiz Vue.js

Zadanie 3 — Testy jednostkowe + raport z testów

1. Wstęp

Quiz Vue.js to jednostronicowa aplikacja (SPA) napisana w Vue 3 (Composition API, `<script setup>`), sprawdzająca wiedzę użytkownika na temat frameworka Vue. Quiz składa się z 7 pytań jednokrotnego wyboru, a po jego zakończeniu wyświetlany jest wynik wraz z oceną.

Aplikacja zbudowana jest z następujących głównych części:

Element	Rola
App.vue	Komponent-dyrygent: zarządza stanem quizu (aktualny ekran, indeks pytania, liczba poprawnych odpowiedzi) i przełącza ekrany.
EkranStart.vue	Ekran startowy z przyciskiem rozpoczynającym quiz (emituje zdarzenie start).
EkranPytanie.vue	Ekran pojedynczego pytania: wyświetla treść i odpowiedzi, obsługuje wybór odpowiedzi, pasek postępu oraz przycisk zakończenia quizu (emituje odpowiedziano, nastepne, zakoncz).
EkranWynik.vue	Ekran wyniku: oblicza procent poprawnych odpowiedzi, dobiera ocenę słowną i wyświetla animację konfetti przy dobrym wyniku (emituje restart).
questions.js	Plik z danymi - tablica pytań (treść, lista odpowiedzi, indeks poprawnej odpowiedzi).

Przepływ aplikacji:

```
EkranStart --(start)--> EkranPytanie --(odpowiedziano / nastepne / zakoncz)--> EkranWynik --(restart)--> EkranStart
```

Główna logika podlegająca testom to: obliczanie wyniku i procentu (`computed`), dobór oceny na podstawie progów punktowych, przekazywanie danych przez props oraz komunikacja między komponentami za pomocą zdarzeń (`emit`).

2. Zakres testów

Testy jednostkowe obejmują logikę aplikacji Quiz — przede wszystkim obliczenia, walidację danych oraz komunikację między komponentami. Sprawdzane są następujące obszary:

- obliczanie wyniku i procentu poprawnych odpowiedzi (`computed` w komponencie EkranWynik),
- dobór oceny słownej na podstawie progów punktowych (100% / co najmniej 80% / co najmniej 60% / poniżej 60%),
- przekazywanie danych do komponentów przez props (treść pytania, numer pytania, liczba pytań),

- komunikację między komponentami przez zdarzenia (emit: start, odpowiedziano, następne, zakoncz, restart),
- blokadę ponownego wyboru odpowiedzi na to samo pytanie,
- zmianę tekstu przycisku nawigacji ("Następne pytanie" kontra "Zobacz wyniki"),
- poprawność danych w pliku `questions.js` (każde pytanie ma odpowiedzi oraz poprawny indeks).

Poza zakresem testów (świadomie pominięte):

- animacja konfetti rysowana na elemencie canvas (logika graficzna, niezwiązana z wynikiem quizu),
- warstwa wizualna (style CSS, kolory, rozmieszczenie elementów),
- testy E2E i integracyjne — raport dotyczy wyłącznie testów jednostkowych.

3. Środowisko

Testy zostały uruchomione w następującym środowisku:

Narzędzie / biblioteka	Wersja
System operacyjny	macOS (Darwin 25.5)
Node.js	24.14.1
npm	11.11.0
vue	3.5.31
vite	7.3.1
@vitejs/plugin-vue	6.0.5
vitest	4.1.8
@vue/test-utils	2.4.10
@vitest/coverage-v8	4.1.8
jsdom	29.1.1

Konfiguracja testów:

- silnik testowy: Vitest (zintegrowany z Vite),
- środowisko uruchomieniowe: jsdom — symulacja przeglądarki (DOM) potrzebna do montowania komponentów Vue,
- montowanie i sprawdzanie komponentów: `@vue/test-utils`,
- pomiar pokrycia kodu: `@vitest/coverage-v8` (raport tekstowy oraz HTML),
- globalne API testowe (`describe`, `it`, `expect`) włączone przez opcję `globals: true` w `vite.config.js`.

Komendy uruchamiające testy:

- `npm run test:run` — jednorazowe uruchomienie wszystkich testów,
- `npm run coverage` — uruchomienie testów wraz z raportem pokrycia kodu.

4. Strategia testowania

Przyjęto strategię testów jednostkowych skupionych na logice aplikacji, a nie na warstwie wizualnej. Każdy komponent testowany jest w izolacji, a komponent nadrzędny App.vue dodatkowo w ujęciu integracyjnym (współpraca z komponentami potomnymi).

Główne założenia:

- każdy test zbudowany jest wg schematu **AAA** (Arrange – Act – Assert): przygotowanie danych, wykonanie akcji, sprawdzenie wyniku,
- komponenty montowane są funkcją mount z biblioteki @vue/test-utils,
- właściwości computed sprawdzane są przez wyrenderowany tekst komponentu (wrapper.text()),
- dane wejściowe przekazywane są przez props, a komunikacja sprawdzana przez wrapper.emitted(),
- interakcje użytkownika symulowane są metodą trigger('click'),
- nazwy testów są **opisowe** — czytają się jak dokumentacja (np. "po wyborze poprawnej odpowiedzi emituje 'odpowiedziano' z wartością true"),
- zgodnie z zasadą "8 dobrych testów ponad 30 słabych" testowane są najważniejsze przepływy, a nie każdy szczegół,
- docelowe pokrycie kodu ustalono na co najmniej 70% linii (wymóg zadania).

Podział testów wg kategorii wymaganych w zadaniu:

- **logika / computed** — obliczanie wyniku i procentu, dobór oceny, liczenie poprawnych odpowiedzi, walidacja danych,
- **props** — przekazywanie pytania, numeru i liczby pytań do komponentów,
- **emit** — zdarzenia start, odpowiedziano, następne, zakoncz, restart,
- **zdarzenia** — kliknięcia przycisków, blokada ponownego wyboru odpowiedzi, obsługa okna dialogowego.

5. Lista przypadków testowych

Poniższa tabela zawiera wszystkie napisane przypadki testowe (28 testów jednostkowych oraz 1 test E2E). Wszystkie testy przechodzą.

Lp.	Plik	Nazwa testu (opis)	Kategoria	Wynik
1	questions.js	eksportuje niepustą tablicę pytań	logika/dane	przeszedł
2	questions.js	każde pytanie ma treść, listę odpowiedzi i indeks poprawnej odpowiedzi	logika/dane	przeszedł
3	questions.js	indeks poprawnej odpowiedzi mieści się w zakresie listy odpowiedzi	logika/dane	przeszedł
4	questions.js	każde pytanie ma unikalne id	logika/dane	przeszedł
5	EkranWynik.vue	oblicza procent poprawnych odpowiedzi (5 z 7 daje 71%)	computed	przeszedł
6	EkranWynik.vue	zaokrągla procent do liczby całkowitej (2 z 3 daje 67%)	computed	przeszedł
7	EkranWynik.vue	pokazuje ocenę "Doskonale" przy wyniku 100%	computed	przeszedł
8	EkranWynik.vue	pokazuje ocenę "Bardzo dobrze" przy wyniku co najmniej 80%	computed	przeszedł
9	EkranWynik.vue	pokazuje ocenę "Warto powtórzyć materiał" przy wyniku poniżej 60%	computed	przeszedł

Lp.	Plik	Nazwa testu (opis)	Kategoria	Wynik
10	EkranWynik.vue	emituje zdarzenie "restart" po kliknięciu przycisku	emit	przeszedł
11	EkranStart.vue	renderuje przycisk rozpoczęcia quizu	render	przeszedł
12	EkranStart.vue	emituje zdarzenie "start" po kliknięciu przycisku	emit	przeszedł
13	EkranPytanie.vue	renderuje treść pytania oraz wszystkie odpowiedzi	props	przeszedł
14	EkranPytanie.vue	wyświetla licznik pytań na podstawie propsów (Pytanie 1 / 7)	props	przeszedł
15	EkranPytanie.vue	po wyborze poprawnej odpowiedzi emituje "odpowiedziano" z wartością true	emit	przeszedł
16	EkranPytanie.vue	po wyborze błędnej odpowiedzi emituje "odpowiedziano" z wartością false	emit	przeszedł
17	EkranPytanie.vue	blokuje ponowny wybór odpowiedzi (drugie kliknięcie nie emituje)	zdarzenia	przeszedł
18	EkranPytanie.vue	pokazuje napis "Następne pytanie", gdy to nie jest ostatnie pytanie	props	przeszedł
19	EkranPytanie.vue	pokazuje napis "Zobacz wyniki", gdy to jest ostatnie pytanie	props	przeszedł
20	EkranPytanie.vue	emituje "nastepne" po udzieleniu odpowiedzi i kliknięciu przycisku	emit/zdarzenia	przeszedł
21	EkranPytanie.vue	po potwierdzeniu w oknie dialogowym emituje zdarzenie "zakonczone"	emit/zdarzenia	przeszedł
22	App.vue	na starcie wyświetla ekran startowy	logika	przeszedł
23	App.vue	po zdarzeniu "start" przełącza na ekran pytania	logika/emit	przeszedł
24	App.vue	po przejściu wszystkich pytań wyświetla ekran wyniku	logika	przeszedł
25	App.vue	zlicza tylko poprawne odpowiedzi (wszystkie poprawne daje komplet)	logika	przeszedł
26	App.vue	zlicza częściowy wynik (3 poprawne odpowiedzi z 7)	logika	przeszedł
27	App.vue	zdarzenie "restart" wraca do ekranu startowego	logika/zdarzenia	przeszedł
28	App.vue	zdarzenie "zakonczone" z ekranu pytania przerywa quiz i pokazuje wynik	logika/emit	przeszedł
29	e2e/quiz.spec.js	główny przepływ quizu: od ekranu startowego do ekranu wyniku (E2E)	E2E	przeszedł

6. Wyniki ilościowe

Metryka	Wartość
Liczba plików z testami jednostkowymi	5
Liczba testów jednostkowych	28

Metryka	Wartość
Testy jednostkowe zakończone sukcesem	28 (100%)
Testy jednostkowe nieudane	0
Testy E2E (Playwright)	1 (przeszedł)
Łączny czas testów jednostkowych	ok. 0,7 s
Czas testu E2E	ok. 3,5 s

Wszystkie testy uruchamiane są komendą `npm run test:run` i kończą się powodzeniem (kod wyjścia 0).

7. Pokrycie kodu

Pokrycie kodu zmierzono narzędziem `@vitest/coverage-v8` (komenda `npm run coverage`). Z pomiaru wyłączono plik startowy `main.js` oraz nieużywany szablonowy plik `stores/counter.js` (wyjaśnienie w sekcji 8).

Metryka	Pokrycie	Pokryte / wszystkie
Lines (linie)	95,41%	104 / 109
Statements (instrukcje)	94,44%	119 / 126
Branches (gałęzie)	94,02%	63 / 67
Functions (funkcje)	92,00%	23 / 25

Pokrycie linii wynosi **95,41%**, co znacznie przekracza wymagane minimum 70%.

Zrzut ekranu z raportu HTML (`coverage/index.html`):

All files

94.44% Statements 119/126 94.02% Branches 63/67 92% Functions 23/25 95.41% Lines 104/109

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
src	100%	17/17	100%	15/15
src/components	93.57%	102/109	92.85%	89/94

Pokrycie w podziale na foldery:

- `src` (App.vue) — 100% linii,
- `src/components` (EkranStart, EkranPytanie, EkranWynik) — 94,68% linii.

8. Zaobserwowane problemy

W trakcie pisania testów napotkano kilka kwestii, które rozwiązano w sposób opisany poniżej (zgodnie z zasadą szczerości w raporcie):

1. Brak obsługi elementu `canvas` w `jsdom`.

Komponent `EkranWynik` przy dobrym wyniku (co najmniej 60%) uruchamia animację konfetti rysowaną na elemencie `<canvas>`. Środowisko `jsdom` nie implementuje metody `getContext` ani pętli animacji (`requestAnimationFrame`), przez co montowanie komponentu wyrzucało błąd. Rozwiązanie: w testach podmieniono te API na atrapy (proste funkcje bez działania).

Dzięki temu możliwe jest przetestowanie logiki wyniku i oceny, a sama animacja graficzna — jako element czysto wizualny — została świadomie pominięta.

2. Pliki wyłączone z pomiaru pokrycia.

Z pomiaru pokrycia wyłączono dwa pliki: `main.js` (punkt startowy aplikacji, który jedynie montuje aplikację w przeglądarce — jego testowanie jednostkowe nie ma wartości) oraz `stores/counter.js` (przykładowy, nieużywany plik wygenerowany automatycznie z szablonu `Vue/Pinia`). Pozostawienie ich zaniżałoby wynik pokrycia bez realnej korzyści.

3. Elementy nieobjęte testami.

Świadomie nie testowano warstwy wizualnej (style CSS, kolory, animacje) oraz pojedynczych linii związanych z zamykaniem okna dialogowego i klatkami animacji konfetti. Są to fragmenty o charakterze wyłącznie prezentacyjnym, których testy jednostkowe nie wnoszą wartości merytorycznej.

9. Wnioski

Aplikacja `Quiz Vue.js` została objęta zestawem 28 testów jednostkowych oraz 1 testem E2E, które w całości przechodzą. Testy pokrywają wszystkie wymagane kategorie: logikę (computed/funkcje), props, emit oraz obsługę zdarzeń.

- pokrycie linii kodu wynosi 95,41% i znacznie przekracza wymagane 70%,
- kluczowa logika aplikacji (liczenie wyniku, dobór oceny, przełączanie ekranów, blokada ponownej odpowiedzi) jest w pełni przetestowana,
- wydzielenie logiki do osobnych komponentów ułatwiło testowanie — każdy ekran można sprawdzić niezależnie,
- dodatkowy test E2E w Playwright potwierdza, że cały przepływ aplikacji działa poprawnie w prawdziwej przeglądarce,
- napotkane problemy (canvas w jsdom) rozwiązano przez podmianę API na atrapy, bez rezygnacji z testowania logiki.

Wniosek końcowy: aplikacja zachowuje się zgodnie z oczekiwaniami, a przyjęta strategia testów daje wysoką pewność poprawności jej działania.

10. Link do repozytorium

Kod aplikacji wraz z testami jednostkowymi, testem E2E oraz niniejszym raportem znajduje się w repozytorium GitHub:

Repozytorium: <https://github.com/KasiaJack/quiz-app>

Demo online: <https://kasiajack.github.io/quiz-app/>